Typed Repositories — Drizzle-inferred contracts for data access

SPEC-DB-002 v0.1 DRAFT Jeremie Nunez

1 Context

Repositories are the only code in the system allowed to issue SQL. They sit directly above `@interview/db-schema` and expose typed, intention-named functions to services. Thalamus cortices and the Sweep pipeline both consume these repositories; keeping them thin, typed, and query-only is what lets the service layer stay pure and testable.

2 Goals

- Define the repository contract: *queries only*, no business logic, no cross-entity aggregation beyond SQL joins.
- Parameter and return types come from Drizzle's `$inferSelect / $inferInsert`; nothing hand-rolled.
- Ban `any` and `unknown` in repository signatures. Payload narrowing happens above the repo, not inside it, and not at its edge.
- Make each repository method easy to mock at the service boundary.

3 Non-Goals

Non-Goal

Repositories do not orchestrate multi-step workflows, call external services, emit events, implement retry policies, or enforce business rules (tier gating, permission checks, quota). They do not serialize to DTOs. That is the service layer's job — "repo = queries, service = logic". Transaction composition spanning multiple repositories is owned by the service.

4 Invariants

Invariant

No `any`, no `unknown`, no `Record<string, unknown>` in repository public signatures. Parameters and return types are either Drizzle- inferred row types, narrow DTO-less tuples, or primitives.

Invariant

A repository method performs at most one logical database operation: a single `select` (with joins allowed), a single `insert`, a single `update`, or a single `delete`. Composition of operations lives in the service.

Invariant

Repositories take the `Database` handle (or a `tx`) by constructor injection. They do not import a global client. This keeps them trivially testable and transaction-safe.

Invariant

JSONB columns are not narrowed inside the repository. They are returned with their schema-declared type; services are responsible for parsing via Zod or equivalent before using them as domain objects.

5 Interface

5.1 Construction

```
import type { Database } from "@interview/db-schema";

export class ResearchFindingRepo {
  constructor(private readonly db: Database) {}
  // ... methods below
}
```

5.2 Representative signatures

```
import {
  researchCycle, researchFinding, researchEdge,
  type ResearchCycle, type NewResearchCycle,
  type ResearchFinding, type NewResearchFinding,
  type ResearchEdge, type NewResearchEdge,
} from "@interview/db-schema";

export class ResearchCycleRepo {
  constructor(private readonly db: Database) {}

  findById(id: bigint): Promise<ResearchCycle | null>;
  listByStatus(status: string, limit: number): Promise<ResearchCycle[]>;
  insert(row: NewResearchCycle): Promise<ResearchCycle>;
  updateStatus(id: bigint, status: string): Promise<void>;
}

export class ResearchFindingRepo {
  constructor(private readonly db: Database) {}

  findByEntity(
    entityName: string,
    entityType: string,
  ): Promise<ResearchFinding[]>;

  findByCycle(cycleId: bigint): Promise<ResearchFinding[]>;
  insertMany(rows: NewResearchFinding[]): Promise<void>;
}

export class ResearchEdgeRepo {
  constructor(private readonly db: Database) {}

  neighbors(
    name: string,
    type: string,
  ): Promise<ResearchEdge[]>;

  insert(row: NewResearchEdge): Promise<ResearchEdge>;
}
```

5.3 What is explicitly forbidden

```
// FORBIDDEN: leaks `any`
findSomething(input: any): Promise<any>;

// FORBIDDEN: leaks `unknown` payloads at the edge
getMeta(id: bigint): Promise<unknown>;

// FORBIDDEN: business logic in the repo
async publishArticleAndNotify(id: bigint) { /* no */ }

// FORBIDDEN: JS-side aggregation when SQL can do it
async listFindingsThenCountByType(cycleId: bigint) {
  const rows = await this.db.select()...;
  return rows.reduce(/* no -- push to SQL */);
}
```

6 Scenarios

Scenario: Service composes two repository calls in one transaction

Given `ResearchCycleRepo` and `ResearchFindingRepo`
When the service wraps `updateStatus` + `insertMany` inside `db.transaction(async (tx) => ...)`
Then both repos accept the `tx` handle as their `Database`
And a failure in either call rolls back the whole transaction.

Scenario: Lookup uses the declared index

Given `idx_research_finding_entity` from SPEC-DB-001 exists
When `ResearchFindingRepo.findByEntity("X","item")` is called
Then the generated SQL filters on `(entity_name, entity_type)`
And `EXPLAIN` reports an index scan, not a sequential scan.

Scenario: Unknown payload stays opaque at the boundary

Given `ResearchCycle.plan` is `jsonb`
When a caller reads `cycle.plan`
Then the type is the schema-declared type (not `any`)
And the service — not the repo — validates and narrows it before use.

Scenario: Bulk insert preserves row shape

Given n `NewResearchFinding` values
When `insertMany(rows)` is called
Then a single SQL `INSERT` is issued
And TypeScript rejects any row missing a `notNull` column.

Scenario: Mocked repo in a service unit test

Given a service under test whose only dependency is `ResearchFindingRepo`
When the test substitutes a hand-rolled mock conforming to the repo's public type
Then no database connection is required
And TypeScript enforces the mock implements every public method.

7 Acceptance Criteria

Acceptance Criterion AC-1

A lint rule (or grep-based test) finds zero `any`, `unknown`, or `Record<string, unknown>` in exported repository signatures under `packages/db-schema/src/repos/`.

Acceptance Criterion AC-2

Every repository method's parameter and return types reference `$inferSelect` or `$inferInsert` from `@interview/db-schema`, or primitives (`bigint`, `string`, `number`), or arrays of those.

Acceptance Criterion AC-3

No repository method contains control flow beyond a single await of a Drizzle query builder expression. A static check (AST rule) enforces one logical operation per method.

Acceptance Criterion AC-4

Repositories accept a `Database | Transaction` handle at construction. A test composes two repo calls inside a transaction and asserts rollback on failure.

Acceptance Criterion AC-5

`ResearchFindingRepo.findByEntity` uses `idx_research_finding_entity`; an integration test runs `EXPLAIN` and asserts an index scan.

Acceptance Criterion AC-6

A service unit test mocks every repo method from a single TypeScript interface derived from the concrete class — and compiles.

Acceptance Criterion AC-7

Attempting to insert a row missing a `notNull` column fails at compile time in a fixture test.

Acceptance Criterion AC-8

No repository imports from `../services/`, `../controllers/`, or any module outside `@interview/db-schema`. Enforced by an import-path lint rule.

8 Traceability

AC	Test file (planned)	Test name
AC-1	`packages/db-schema/tests/repos/no-any.spec.ts`	`repo signatures ban any/unknown`
AC-2	`packages/db-schema/tests/repos/types.spec.ts`	`repo signatures use inferred types`
AC-3	`packages/db-schema/tests/repos/single-op.spec.ts`	`one logical query per method`
AC-4	`packages/db-schema/tests/repos/tx.int.spec.ts`	`rollback across two repos`
AC-5	`packages/db-schema/tests/repos/finding-index.int.spec.ts`	`findByEntity uses entity index`
AC-6	`packages/db-schema/tests/repos/mockability.spec.ts`	`repo is mockable from its class type`
AC-7	`packages/db-schema/tests/repos/insert-required.spec.ts`	`missing notNull column is compile er`
AC-8	`packages/db-schema/tests/repos/isolation.spec.ts`	`repos have no upward imports`

9 Open Questions

- Should repositories expose a cursor/stream API for large result sets (e.g. all findings for a long-running cycle), or is pagination enough?
- Do we introduce a `Repo` base class/interface for consistent `tx` propagation, or keep each class free-standing?
- Should JSONB columns be narrowed at the repo boundary via a branded type (still no `any`), or left to the service with Zod?
- How do we express "upsert" semantics without violating the one-operation-per-method invariant — dedicated method, or service composition of `findById` + `insert`/`update`?